

Task Scheduling Stabilization for Solar Energy Harvesting Internet of Things Devices

Antonio Caruso¹, Stefano Chessa², Soledad Escolar³, Fernando Rincón³, Juan C. López³

¹*Dept. of Mathematics and Physics "Ennio de Giorgi", University of Salento, Lecce, Italy*

²*Computer Science Department, University of Pisa, Pisa, Italy*

³*School of Computing Science, University of Castilla-La Mancha, Ciudad Real, Spain*

Abstract—Energy neutrality of Internet of Things devices powered with energy harvesting is a concept introduced to let these devices operate uninterruptedly. A method to achieve it is by letting the device scheduling different tasks characterized by different energy costs (and quality), depending on the current energy production of the energy harvesting subsystem and on the residual battery charge. In this context, we propose a novel scheduling problem that aims at keeping the energy neutrality of the scheduling while maximizing the overall quality of the executed tasks and minimizing the leaps of quality among consecutive tasks, so to improve the stability of the output of the device itself. We propose for this problem an algorithm based on a dynamic programming approach that can be executed even on low-power devices. By simulation we show that, with respect to the state of the art, the scheduling by our algorithm greatly improve the stability of the device with a minor penalty in terms of overall quality.

Index Terms—Solar Energy Harvesting; Multi-objective Scheduling Problem; Stability of Scheduling; Internet of Things.

I. INTRODUCTION

It is widely accepted that energy is one of the major constraints to the perpetual operation of Internet of Things (IoT) devices. These devices are usually not connected to the mains but are fed by means of some energy storage (e.g. a battery) able to sustain their circuitry and operation and thus, their lifetimes depend directly of the battery charge. Even low-power devices, as the ones used in multiple outdoor IoT domains (e.g. smart cities, smart agriculture, smart environment) perform sensing, processing, and communications tasks that demand a (minimal) energy consumption and are, therefore, affected from this limitation.

To the end of extending the operation of the devices, energy-harvesters (e.g. solar cells, wind turbines) may be attached to the devices and provide them electricity from a clean, cheap, and renewable source of energy found naturally in the environment, as the Sun, the wind, vibrations or movement. In particular, solar power is widely used in IoT applications due to its great power density and to the reliability of solar panels. However, because of the uncontrollable nature of the solar energy, a solar-based energy-harvesting system is not sufficient to provide perpetual operation of the IoT device: there is no guarantee about when (i.e. which moment of time) and how much (i.e. the amount) of energy may be produced. Thus, it is needed to complement such systems with some strategy aimed at achieving uninterrupted execution. To this purpose Kansal et al. [1] introduced the concept of energy neutrality, that consists

in modulating the load of the device in such a way that it equals the energy production during a given time interval (in the case of solar energy harvesting devices this time interval is usually one day). In that work, Kansal et al. proposed to achieve energy neutrality in a device by considering the daily forecast of energy production and by modulating the duty cycle of the device, which could be increased in case of an excess of energy production or decreased in case of energy underproduction.

Following the concept of energy neutrality, several other works [2]–[7] addressed this problem by proposing more complex models in which the load can be modulated by executing different tasks characterized by a quality and an energy cost. With such models energy neutrality can be achieved by using a scheduling algorithm that solves an optimization problem to find a schedule of the tasks that maximizes the sum of their qualities under the constraint of energy neutrality. However, since this scheduling problem is NP-Hard, the earlier approaches adopted simple greedy strategies. More recently the work in [8] showed that the problem can be solved even on very low-power devices by using a dynamic programming approach that finds the optimum schedule.

In this work we reconsider this latter work to solve a problem of stability in the scheduling that arises when using the above mentioned optimizer. In fact, the optimum schedule that is reached may, in general, include large leaps in quality between consecutive tasks. As these leaps in quality are achieved by rather different operations of tasks in terms of duty cycle, data processing and even in the use of different sensors, the output of the device for the end-user results rather variable and unstable, making it thus difficult to manage.

We thus solve this problem by introducing a multi-objective scheduling problem, in which the first objective is the maximization of the overall tasks quality and the second objective is the minimization of the leaps among consecutive tasks. In this form however the problem becomes feasible if the optimization is run on a powerful machine (e.g. on a cloud application controlling the IoT device, or at the edge/fog level). In order to enable the execution of the scheduler even on a low-power device, we propose a simpler version of the problem in which the two objectives are combined into one single objective that takes into account both needs (maximization of quality and minimization of leaps), and we show how this problem can be solved efficiently by using a dynamic programming algorithm, even on very low power IoT devices. By means of simulation we also show that, as compared to

the approach of [8], our approach drastically limits the leaps of quality among consecutive tasks thus making the system much more stable, at the cost of a rather limited loss in the overall quality of the scheduling.

The rest of the paper is organized as follows: Section II presents the state of the art, sections III and IV introduce the optimization problem and the novel algorithm, respectively, Section V presents the simulation results and Section VI draws the conclusions and future work.

II. BACKGROUND AND RELATED WORKS

An energy-harvesting system is comprised of four major elements: a *harvester*, a *power converter*, a *load*, and an *energy storage*. A solar-based harvester consists in a photovoltaic (PV) module that uses the sunlight as a source of energy and convert it into electricity by means of the photovoltaic effect. The output power that a solar panel is able to extract depends on the irradiance D (i.e. density of incident power on the surface), the surface S and the efficiency μ (i.e. the percentage of conversion of the irradiance into electrical power). A power converter is an electronic device that regulates the power passing from the PV panel to other output devices avoiding that both the output voltage and the current suffer fluctuations not controlled that may cause damages to the target device. The *load* is the computational device connected to the system that demands energy to carry out its operations. It is normally composed by a MCU (Micro-Controller Unit), one or several radios for data transmission and reception, one or several sensors, and an external memory for permanent data storage. Finally, an energy-storage will provide a buffer of energy able to maintain the operation of the load under periods of low or null energy production. Traditionally, the energy harvesting systems have included rechargeable batteries as energy storing device. More recently, they have started to include capacitors or super-capacitors as primary or secondary energy storage, fundamentally, because of their longer lifetimes and larger range of operating temperatures.

However, even if powered with an energy harvesting system, an IoT device may still experience energy shortages due to the non-controllable nature of the solar energy. To this purpose, Kansal [1] introduced the energy neutrality condition, which states that given a certain reference time period, the battery level at the end of the period is always greater than or equal to the battery level at the beginning of the same period. To satisfy this condition, there exist four major strategies: Dynamic Voltage and Frequency Scaling (DVFS), Duty Cycle Adaptation, Sensing and Sampling Rate Adaptation and Task Scheduling. A review of these four techniques are described in [9], [10].

Specifically, we have contributed to the last category with several works that propose algorithms devoted to find the optimal scheduling that maximizes the utility of the applications running on top of the device while the energy neutrality condition is kept, and a real testbed for them [2], [11]. In this context, an application is described in terms of tasks, where each task is implemented by alternative versions with a certain utility degree, where the higher the utility, the higher

the energy consumption. For example, the task of sampling of a sensor can be done with more or less frequency and accuracy. On the base of this model, we have considered progressively richer scenarios, from a simple one with one energy harvesting device [6], [7] to scenarios involving more devices connected to a gateway or sink, that also has energy harvesting capability [3]–[5]. In a recent work [8] we introduced a scheduling algorithm based on dynamic programming that finds the optimum scheduling, and we proved that such an approach is even feasible in low-end devices by considering the limited resolution of these devices in sampling the battery charge and the power production of the photovoltaic panel.

Similarly to [8], other works have addressed the problem of optimization of the tasks scheduling under different scenarios. For example, in [12] a mixed-integer linear programming (MILP) formulation is devoted to the optimization of the quality of security of the tasks scheduled under energy-constraints on a battery-powered MultiProcessor SoC (MPSoC). The security may be achieved by a set of cryptographic algorithms, each one with a time overhead and an energy consumption, and a quality associated to the service. The experimentation results show how the algorithm proposed saves energy and improves quality of security for MPSoC compared to other approaches. In [13] the IoT devices, powered by renewable energy, make a computational offloading using resource-rich fog servers, which are powered by grid electricity, in the cloud and propose a strategy for optimization of real-time tasks executed on the fog aimed at maximizing both the system QoS and the QoS of individual real-time applications. The experimental results demonstrate an improvement of the overall and individual application QoS by up to 101.93% and 59.30%, respectively. The work in [14] describes an approach in two-stages for real-time electricity pricing and scheduling. To this end, an optimization strategy for IoT device scheduling modeled as a 0-1 Knapsack problem is proposed (i.e. the device can or not be scheduled). The authors leverage into our work in [15] to define an energy-harvesting environment where the nodes may also buy electricity when not enough energy is provided from the harvester. The results demonstrate that the accepted requests ratio of the the energy-harvesting scenario is up to 20% greater than the ratio of the scenario where the energy harvesting is not applied, due to the fact that the energy consumption of a device per time slot is reduced by the harvested energy, and thus more requests can be accepted.

Differently from the cited works, in this paper we pursue a different objective, based on the graceful degradation of the quality of the scheduled tasks, while the energy neutral condition is still guaranteed. This approach could be a preferred choice from the user point of view, by providing a more progressive degradation of the behaviour and quality of the tasks executed on the device along the time.

III. OPTIMIZATION MODEL OF ENERGY HARVESTING DEVICES

We assume low-power IoT device equipped with a micro-controller, memory, wireless interface and a set of sensors/actuators. We also assume that it is powered by an energy

harvesting subsystem comprising a rechargeable battery (of maximum capacity B_{max}) and a solar panel. We also assume that the minimum charge that lets the device to operate is B_{min} and we denote with η the charging efficiency of the battery (this accounts for loss of energy due to its conversion and battery charging process).

In order to optimize the use of the battery and thus to achieve energy neutrality, the device is programmed in order to modulate the load (that is, its energy consumption), so to consume, within a predefined time frame, the same amount of energy that it produces. Considering that the solar energy has a natural cycle of 24 hours, we take this time frame to be exactly of 24 hours.

In order to modulate the load of the device, we adopt the same approach of [8]. In particular we assume that the device is preloaded with a set of N alternative tasks $\mathcal{T} = \{t_1, \dots, t_N\}$ each characterized by a different energy consumption $c_i, i \in [1, N]$ and quality of service $q_i, i \in [1, N]$. For example, different tasks may implement the functionality of the device sampling at different frequencies, or they may execute a lighter/more complex data aggregation or they may communicate less/more frequently with the base station. It is important to stress that in our model the tasks are alternative, that is, they all implement an equivalent functionality but with different performances (and thus with different quality of service and energy consumption).

Still to the purpose of load modulation, the device also embeds a scheduler that selects the tasks to be executed in order to guarantee the energy neutrality of the device. We highlight here, that even in IoT nodes that are really constrained, with a low amount of memory or computational resource, it is possible to build a very small scheduler, see for example FUSIX¹ a small Unix like OS implementation for the Raspberry Pico.

In particular, the scheduler assumes that the 24 hours time frame is split into K time slots of the same duration $24h/K$ and, at the beginning of each day, it produces an energy neutral assignment of the tasks to each slot based on the expected energy production in each slot. Considering that the expectation on the energy production in each slot may not be met, the scheduler may be ran again to recompute the assignment of the tasks to the slot if the device is deviating from the energy neutrality target. Note that the scheduler executes an optimization: on one hand it has to respect the energy neutrality constraint, while on the other it has to maximize the overall quality of the assignment.

For each $i \in [1, K]$, let e_i be the expectation for the energy to be produced in slot i , B_i be the energy charge of the battery at the beginning of slot i , B_{K+1} be the battery charge at the end of the last slot, and let x_{ij} be a Boolean variable which is $x_{ij} = 1$ iff task t_j (with $j \in [1, N]$) is assigned by the scheduler to slot i . Given assignment x_{ij} for all $i \in [1, K]; j \in [1, N]$, we denote with $\hat{q}_i$ the quality of the task assigned to slot i and with $\hat{c}_i$ the energy consumption of the task assigned to slot i . Hence : $\hat{q}_i = \sum_{j=1}^N x_{ij} q_j$ and $\hat{c}_i = \sum_{j=1}^N x_{ij} c_j$.

On this base, and considering that the battery charge cannot exceed B_{max} , the expected battery charge at the beginning of a generic slot $i+1$ can be computed as the minimum between B_{max} and the battery charge at the beginning of the slot i plus the amount of energy that is produced in excess to the consumption minus the energy that is consumed in excess to the production, that is: $B_{i+1} = \min \{B_{max}, B_i + \eta[e_i - \hat{c}_i]^+ - [\hat{c}_i - e_i]^+\}$, and the constraint of energy neutrality is expressed as $B_{K+1} \geq B_1$.

In [8] the optimization problem of the scheduler has been formulated as an integer linear programming problem, where the only objective is to maximize the overall quality of service of the device. However, as already discussed in the introduction, this approach is not fully satisfactory because it may also result in drastic changes of QoS between the tasks assigned to consecutive slots, thus making the overall behaviour of the device rather unstable from a practical point of view. Instead, here we seek for an optimization that tends to moderate any degradation of the QoS among consecutive slots by introducing an additional optimization objective. The consequence is that the problem of the scheduler now becomes a multi-objective optimization problem that can be formalized as follows:

Problem 1 (Multi-objective Stable plan Scheduling):

$$\text{maximize } z = \sum_{i=1}^K \hat{q}_i \quad (1)$$

$$\text{minimize } v = \max_{i \in [1, K]} |\hat{q}_i - \hat{q}_{i+1}| \quad (2)$$

$$x_{ij} \in \{0, 1\} \quad \forall i \in [1, K], j \in [1, N] \quad (3)$$

$$\sum_{j=1}^N x_{ij} = 1 \quad (4)$$

$$B_{i+1} = \min \left\{ B_{max}, B_i + \eta[e_i - \hat{c}_i]^+ - [\hat{c}_i - e_i]^+ \right\} \quad \forall i \in [1, K] \quad (5)$$

$$B_{min} \leq B_i \quad \forall i \in [1, K] \quad (6)$$

$$B_1 \leq B_{K+1} \quad (7)$$

where $[x]^+ = \max(x, 0)$

In the problem there are two objective functions: (1) is the sum of the quality of the tasks assigned to the slots, and should be maximized, while (2) express the requirement that the sum of loss of the quality of service from each slot to the next should be minimized. For what concerns the constraints, (3) and (4) express the requirement that exactly one task must be assigned per slot, (5) expresses the constraint of the battery charge at the next slot, that depends on the task assignment as already discussed, (6) expresses the requirement that the battery charge should never go below the minimum, otherwise the device would stop working, and finally (7) expresses the energy neutrality constraint.

Although this formulation takes into account the requirement of stability of the scheduling by aiming also at reducing the differences in QoS between consecutive slots, it requires

¹<https://www.raspberrypi.com/news/how-to-get-started-with-fuzix-on-raspberry-pi-pico/>

multi objective optimization and an objective function (2) that is not linear, which is very complex to implement (if ever possible) in a resource constrained IoT device. For this reason we propose an alternative problem which keeps the same constraints, but that replaces the two objective functions with a function with the non-linearity limited only to the absolute value. Specifically, we introduce a penalty term to the sum of the QoS that takes into account the variation of QoS between consecutive slots as follows:

$$\text{maximize } z = \sum_{i=1}^K \hat{q}_i - \gamma(|\hat{q}_i - \hat{q}_{i+1}| - 1) \quad (8)$$

Note that the multiplicative factor γ is used to modulate the impact of the differences between tasks quality with respect to the other goal of maximizing the total quality, the -1 value is introduced to discount the penalty factor when the schedule changes with minimal variation, i.e. when we move from two tasks with minimum quality differences (in particular this factor can be adjusted to be at least the cost of the idle task, or the latter can be normalized to be 1 as we assume in the Simulation Section). In [8] we proved that the problem of *Energy Neutral Optimal Scheduling* is NP-Hard. However, we show that it can be solved by a proper dynamic programming approach even in low-power devices. Clearly this formulation, that is more complex of the original problem remains NP-Hard. The adopted solution in this paper is not particularly costly and it does not vary the order of growth of the computational complexity.

IV. STABLE SCHEDULING ALGORITHM

We propose a dynamic programming algorithm that is inspired to a variation of unbounded knapsack problem, with the major difference that the solution is computed proceeding backwards with respect to the time slot, i.e. we compute a table M of size $K \cdot B_{max}$, where each row is associated to a slot and each column to a level of battery, the value of $M(k, b)$ for $k \in [1, K]$ and $b \in [0, B_{max}]$ is the value of the objective function, i.e. the maximum quality of a schedule that is energy neutral starting with a battery of b and using k slots. The resulting Bellman equations, which adopt a *penalty factor* p , are:

$$\begin{aligned} opt(k, b) &= \max_{i=1 \dots N} \{ q_i + opt(t+1, B'(i)) - p \mid B'(i) \geq B_{min} \} \\ \text{with: } B'(i) &= \min\{B_{max}, b - c_i + e_i\}, p = \gamma(|\hat{q}_i - \hat{q}_{i+1}| - 1) \end{aligned} \quad (9)$$

The Python code with the function `ScheduleWithPenalty` that computes the value (quality) of the optimal solution based on the above recursive equations are in Listing 1, while the function `BuildSchedule` in (2) is used to construct the optimal schedule; in the following we discuss both in details, note that all indexes for array and matrices are zero based. The function `ScheduleWithPenalty` use two matrices: M and I (lines 2,3) initialized with zeros. The role of matrix M has been already explained above, while matrix I is used to keep track of the best task associated to each solution in $M(k, b)$. The function compute the solution from slot $K-1$ to 0 (line 4), for

```

1 def ScheduleWithPenalty(slots, Bstart, Bmin, Bmax, E, Tasks, gamma=0.5):
2     M = np.zeros((slots, Bmax+1), dtype=int)
3     I = np.zeros((slots, Bmax+1), dtype=int)
4     for k in range(slots-1, -1, -1):
5         for B in range(Bmax, -1, -1):
6             qmax, idmax = -100, 0
7             if (k == slots-1):
8                 for t, task in enumerate(Tasks):
9                     if B-task.cost+E[k] >= Bstart and task.quality > qmax:
10                         qmax = task.quality
11                         idmax = t+1
12             else:
13                 for t, task in enumerate(Tasks):
14                     Bprime = min(B-task.cost+Eprod[k], Bmax)
15                     if Bprime >= Bmin:
16                         q = M[k+1][Bprime]
17                         if (q == 0): continue
18                         j = I[k+1][Bprime]-1
19                         penalty = int(gamma * (abs(task.quality -
20                                     Tasks[j].quality) - Tasks[0].quality))
21                         if q + task.quality - penalty > qmax:
22                             qmax = q + task.quality - penalty
23                             idmax = t+1
24             M[i][B] = qmax if qmax != -100 else 0
25             I[i][B] = idmax

```

Listing 1: Python implementation of the Dynamic Programming Scheduler with *index distance penalty*.

each slot we search an optimal solution for all possible battery levels (line 5, from B_{max} to 0). Given the slot k and battery level at the beginning of the slot b , in the last slot the cycle in lines 7-11 find the best task that is energy neutral, i.e. if we start with b we end with B_1 , note that at the end of the loop, we remember the task quality and index and put them in M and I in lines 24-25; we add 1 to the best task index stored in $idmax$ (line 11,23), so that the value of 0 in that variable signal a non-feasible solution. Lines from 13 to 23 repeat the same search for the other slots, in particular we compute the values of $B'(i)$ as in Eq. 9 and use it to 1) check that it satisfy the constraint (6) of the model, 2) compute the penalty factor and the new quality using Equation 8.

The function `BuildSchedule` build the schedule s , with K indexes of tasks: it start with the value of $I[0][Bstart]$ (the index of the best task for the first slot, with initial battery equal to $Bstart = B_1$), compute the residual battery at the beginning of the next slot, check that this level is feasible for B_{min} and updates the iteration variable for the next iteration, filling all values of S . At the end it returns a Python list s with the tasks scheduled in each slot and the quality of this schedule: note, the latter, is computed without taking into account the penalty factor, in order to be comparable with the original algorithm. Note also that the code is not optimized at all for performance (memory or time), but for clarity.

V. SIMULATIONS

We have evaluated the Algorithm described in the previous section by means of a simulator implemented in Python3. In our simulations, we are interested in comparing the performance of our algorithm against that described in [8], which computes the optimal scheduling with the only objective of maximization of the overall quality of the tasks. In addition,

```

1 def BuildSchedule(K,Bmin,Bmax,Bstart,Panel,Tasks):
2     S = [0]*K
3     B = Bstart
4     for i in range(K):
5         S[i] = I[i][B]-1
6         if (S[i] < 0): return (S,0)
7         B = min(B + Panel[i] - Tasks[ S[i] ].cost,Bmax)
8         if B < Bmin: return ([],0)
9     if B < Bstart: return ([],0)
10    return(S,sum(Tasks[s].quality for s in S))

```

Listing 2: Python code that construct the schedule from I, check that everything is feasible and recompute the quality.

Table I

SIMULATOR PARAMETERS: DAY DIVIDED IN 24 HOURS, 10 TASKS WITH COSTS c_i AND QUALITY q_i . E IS THE PRODUCTION OF THE SOLAR PANEL FROM SLOT 8 TO SLOT 21 ZERO OTHERWISE.

K	24
N	10
B_{min}	160 (mAh)
B_1	1800 (mAh)
B_{max}	2000 (mAh)
c_i	[1, 16, 27, 38, 49, 60, 70, 81, 92,103]
q_i	[1, 16, 27, 37, 48, 59, 68, 79, 90,100]
E	[3,45,133,215,285,327,339,322,255, 60, 66, 63, 23,9]

the simulations aim at observing **how our algorithm behaves under different values of penalty and under different sets of tasks**. For the first experiment, we have used the parameters shown in Table I: the number of slots (K), the number of tasks (N), with the corresponding cost c_i and quality q_i , and the minimum, maximum and initial battery level (B_{min} , B_{max} , and B_1 , respectively). We have used a couple of values of $\gamma=\{0.5,1\}$, that result in different penalties and thus different schedules. Note that in all experiments the costs of the tasks are in increasing order, and so it is the quality, so the differences between indexes of two tasks is proportional to the differences in their quality, i.e. we used this approach to simplify the discussion on the simulation outcomes.

Figure 1 shows the results of this first experiment. The output of the run with the optimal algorithm gives the following schedule: $s = [8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 9, 9, 9, 9, 9, 5, 5, 4, 2, 0, 1, 8, 8]$, with a quality = 1880 (average quality of 78,33%), if we compute the *jumps* among the task indexes, we obtain a vector of values $J = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 4, 0, 1, 2, 2, 1, 7, 0]$, two simple statistics are used to measure how 'stable' is the schedule, the sum of all this gaps, and the maximum gap, we see here a sum of 18 and a maximum gap of 7.

The new algorithm presented here provides, for $\gamma=\{0.5,1\}$, the next schedulings: $S_{\gamma=0.5} = [9, 9, 9, 9, 9, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 5, 5, 4, 3, 3, 4, 4, 4]$ and $S_{\gamma=1} = [3, 4, 4, 4]$, with qualities equal to 1874 and 929 (average qualities of 78,08% and 38,38%), respectively, and a sum of 7 and 1 jumps between tasks, respectively (max = 3,1): $J_{\gamma=0.5} = [0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 0, 1, 1, 0, 0]$ and $J_{\gamma=1} = [0, 1, 0, 0]$. As you can see, the jumps in the new solution are clearly smaller with the maximum

jump between consecutive tasks equal to 3 with $\gamma = 0.5$ and 1 with $\gamma = 1$ against a maximum jump of 7 for the optimal algorithm. Figure 2 shows the different jumps. We note that while keeping γ moderate (0.5) decrease the jumps without decreasing the overall quality with respect to the old algorithm, if we increase the penalty factor too much (with values of γ close to 1) the algorithm is too constrained and produce a schedule that is too stable actually, decreasing by a large factor it's quality.

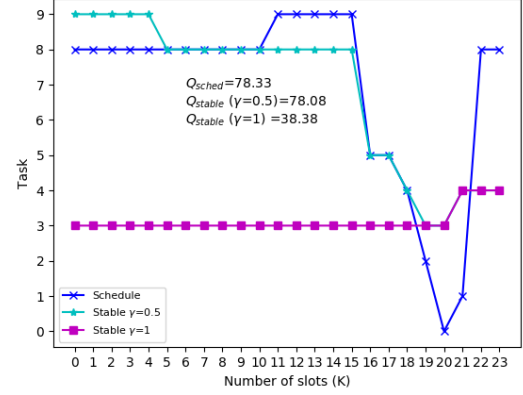


Figure 1. Comparison of the optimal schedule and stable scheduling with different values of γ

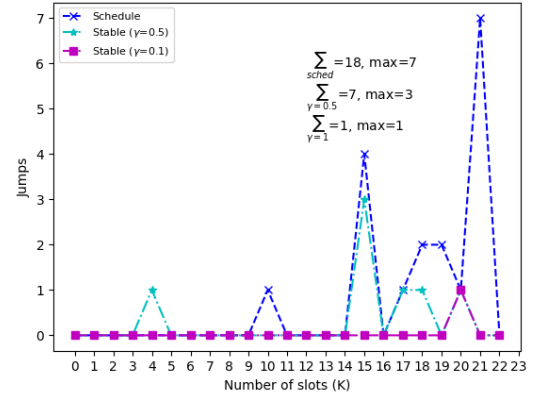


Figure 2. Number of jumps computed for the optimal schedule and stable scheduling with different values of γ

The next experiment consisted in test several sets of tasks to compare the quality obtained by the two algorithms fixing a value of $\gamma=0.5$ and the same energy production per hour (in mA) E as in the first row of table II.

Figure 3 shows the scheduling obtained by the optimal algorithm and the stable algorithm for this set of tasks. As observed, the stable algorithm proposed provides an average quality close to the obtained by the optimal algorithm. However, the sum of jumps done in the optimal version is respectively, for the three sets of tasks 18, 7 and 20, while the jumps done in the stable version are, 7, 6, and 7, which means 65% less jumps in the better case, for a degradation of quality

Table II
SET OF TASKS EVALUATED: COST AND QUALITY

E	[0,...,0,3,45,133,215,285, 327, 339, 322, 255, 60, 66, 63, 23,9,0,0,0]
c_1	[1, 16, 27, 38, 49, 60, 70, 81, 92,103]
q_1	[1, 16, 27, 37, 48, 59, 68, 79, 90,100]
c_2	[1, 16, 24, 37, 41, 53, 60, 71, 82, 95]
q_2	[1, 11, 27, 31, 48, 59, 65, 79, 86,100]
c_3	[1, 16, 27, 38, 49, 60, 70, 81, 92,103]
q_3	[1, 10, 25, 35, 45, 55, 68, 85, 92,100]

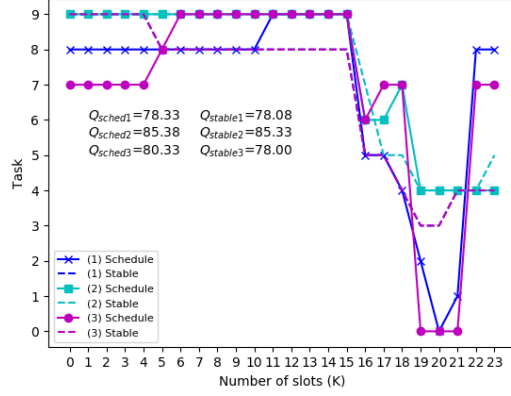


Figure 3. Comparison of the optimal schedule and stable scheduling for the set of tasks evaluated

of only 2.5%. Note that this reduction of jumps impacts on the application functionality, which will be perceived as a more stable behaviour.

VI. CONCLUSIONS AND FUTURE WORK

To our knowledge, the problem of the stability of the behaviour of an IoT device has received no consideration at all in the past works on energy neutrality. We believe instead that, as energy harvesting with energy neutrality in IoT becomes more and more adopted, this problem will become more and more relevant. In particular, it is critical if the IoT device is a sensor, because its output can be greatly affected by continuous fluctuation of its sampling rate or by continuous changes in the way it pre-process its data, which are required by the energy neutrality approaches to continuously adapt the energy load of the device to the actual energy production. It is also relevant if the device is an actuator because its behaviour may change over time due to energy constraints that do not depend on its function, and its controlling agent need to be constantly aware of this. Clearly, with energy neutrality it is not possible to avoid such changes of behaviour of the device, but it is possible to limit their extent in order to force the device to a more stable behaviour, by gracefully degrading or upgrading its performance. Following this observation we have proposed a novel scheduling problem for energy neutral IoT devices and a novel algorithm for its solution. Our simulation results show that, as compared with the state of the art, our approach

greatly reduces the leaps in terms of quality (performance) among consecutive tasks executed by the device, at the cost of a minor penalty in the overall scheduling quality.

As future work we plan to extend this approach to the concept of statistical energy neutrality [15] and to use other cost-utility patterns to represent different IoT use cases and scenarios, as for instance, the ‘utility-like’ concave utility function to characterize diminishing returns.

ACKNOWLEDGMENT

This research has been partially supported by the PRA project AUTENS (Sustainable Energy Autarky) of the University of Pisa, the EU’s Horizon 2020 programme under project SHAPES (GA N° 857159) and Grant TALENT (PID2020-116417RB-C44) funded by MCIN/AEI/10.13039/501100011033.

REFERENCES

- [1] A. Kansal, J. Hsu, S. Zahedi, and M. B. Srivastava, “Power management in energy harvesting sensor networks,” *ACM Trans. Embed. Comput. Syst.*, vol. 6, no. 4, p. 32-es, sep 2007.
- [2] M. Kuzman, X. del Toro Garcia, S. Escolar, A. Caruso, S. Chessa, and J. C. Lopez, “A Testbed and an Experimental Public Dataset for Energy-Harvested IoT Solutions,” in *2019 IEEE 17th International Conference on Industrial Informatics (INDIN)*. IEEE, Jul. 2019.
- [3] S. Escolar, S. Chessa, and J. Carretero, “Energy management of networked, solar cells powered, wireless sensors,” in *Proceedings of the 16th ACM international conference on Modeling, analysis & simulation of wireless and mobile systems*. ACM, 2013, pp. 263–266.
- [4] S. Escolar, S. Chessa, and J. Carretero, “Energy-neutral networked wireless sensors,” *Simulation Modelling Practice and Theory*, vol. 43, pp. 1–15, 2014.
- [5] S. Escolar, S. Chessa, and J. Carretero, “Quality of service optimization in solar cells-based energy harvesting wireless sensor networks,” *Energy Efficiency*, pp. 1–27, 2016.
- [6] S. Escolar, S. Chessa, and J. Carretero, “Optimization of Quality of Service in Wireless Sensor Networks Powered by Solar Cells,” in *IEEE 10th International Symposium on Parallel and Distributed Processing with Applications*, July 2012, pp. 269–276.
- [7] S. Escolar, S. Chessa, and J. Carretero, “Energy management in solar cells powered wireless sensor networks for quality of service optimization,” *Personal and Ubiquitous Computing*, vol. 18, no. 2, pp. 449–464, 2014.
- [8] A. Caruso, S. Chessa, S. Escolar, X. del Toro, and J. C. López, “A Dynamic Programming Algorithm for High-Level Task Scheduling in Energy Harvesting IoT,” *IEEE Internet of Things Journal*, vol. 5, no. 3, pp. 2234–2248, June 2018.
- [9] M. M. Sandhu, S. Khalifa, R. Jurdak, and M. Portmann, “Task scheduling for simultaneous iot sensing and energy harvesting: A survey and critical analysis,” *arXiv: Signal Processing*, 2020.
- [10] P. Loreti, L. Bracciale, and G. Bianchi, “Stablesens: Sampling time decision algorithm for iot energy harvesting devices,” *IEEE Internet of Things Journal*, vol. 6, no. 6, pp. 9908–9918, 2019.
- [11] A. Caruso, S. Chessa, S. Escolar, X. del Toro, M. Kuzman, and J. C. López, “Experimenting forecasting models for solar energy harvesting devices for large smart cities deployments,” in *2019 IEEE Symposium on Computers and Communications (ISCC)*, 2019, pp. 1177–1182.
- [12] J. Zhou, J. Sun, P. Cong, Z. Liu, X. Zhou, T. Wei, and S. Hu, “Security-critical energy-aware task scheduling for heterogeneous real-time mpsoes in iot,” *IEEE Transactions on Services Computing*, vol. 13, no. 4, pp. 745–758, 2020.
- [13] K. Cao, J. Zhou, G. Xu, T. Wei, and S. Hu, “Exploring renewable-adaptive computation offloading for hierarchical qos optimization in fog computing,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 10, pp. 2095–2108, 2020.
- [14] L. Park, C. Lee, J. Kim, A. Mohaisen, and S. Cho, “Two-stage iot device scheduling with dynamic programming for energy internet systems,” *IEEE Internet of Things Journal*, vol. 6, no. 5, pp. 8782–8791, 2019.
- [15] S. Escolar, A. Caruso, S. Chessa, X. d. Toro, F. J. Villanueva, and J. C. López, “Statistical energy neutrality in iot hybrid energy-harvesting networks,” in *2018 IEEE Symposium on Computers and Communications (ISCC)*, 2018, pp. 00444–00449.